

Práctica Unidad 1 - Semana 2 - Metodología de Sistemas II (2026)

TP – | Estudiante: Varela Santiago Octavio - Email Institucional: santiago.varela@tupad.utn.edu.ar

(Comisión 15 - DNI 38011156)

Parte A - Diagnóstico del Diseño

A. Indicar qué principios SOLID se están violando.

El código base, encapsulado en la clase Notificador, presenta al menos las siguientes violaciones:

- **La clase Notificador viola el SRP (Single Responsibility Principle)** al asumir la responsabilidad de manejar la lógica de múltiples canales de comunicación (Email, SMS, WhatsApp). Según el SRP, una clase debe tener una única razón para cambiar. Al tener múltiples tipos de notificaciones, Notificador es sensible a cambios en cada canal.
- **El diseño no respeta correctamente el OCP (Open/Closed Principle)**. Se trata de un diseño cerrado a la extensión y abierto a la modificación. La implementación del método enviar utiliza condicionales (if/else) que requieren que se altere el código de la clase Notificador cada vez que se introduce un nuevo medio de notificación. El sistema no permite añadir funcionalidades sin modificar el código base.
- **No cumple con el DIP (Dependency Inversion Principle)**. Existe una fuerte dependencia entre el código de alto nivel y las clases concretas de bajo nivel (implementaciones de notificación). El sistema no está desacoplado mediante abstracciones, lo que impide que las dependencias sean invertidas hacia interfaces, impidiendo la flexibilidad y la inversión de control.

B. Explicar por qué el diseño actual es frágil.

El diseño es inherentemente frágil debido al acoplamiento fuerte entre Notificador y sus implementaciones concretas. Esto también genera un riesgo de regresión, ya que si se cambia la lógica de un canal (por ejemplo, un cambio en el formato del mensaje de SMS), se pueden generar fallos inesperados en canales que están operando correctamente (como Email), debido a que todos residen en un mismo bloque de código condicional.

C. Describir los problemas de mantenibilidad del código.

- **Mantenimiento Centralizado:** La centralización de la lógica en Notificador crea un punto único de fallo y de complejidad.

- **Baja Cohesión:** El código no está agrupado por responsabilidad (como ya se mencionó y se vio en el TP anterior), sino por una estructura condicional, lo que reduce la coherencia.
- **Complejidad Creciente:** A medida que se añaden más canales, el método enviar se convierte en un monolito (alto acoplamiento) difícil de leer, depurar y entender.

D. Explicar cómo impacta esto en el costo del cambio.

El costo del cambio en este sistema es alto y puede crecer de forma exponencial según la cantidad de canales:

- **Incremento del esfuerzo y costos exponenciales:** Cada nueva funcionalidad (un nuevo canal de notificación) no solo requiere una nueva implementación, sino la modificación del código central existente, aumentando el riesgo de introducir regresiones, lo que genera un riesgo operacional general.
- **Aumento del Alcance de Pruebas:** La necesidad de probar la clase completa en lugar de componentes aislados puede aumentar significativamente el esfuerzo de QA y el tiempo de lanzamiento del software.

Parte B - Aplicación de Principios SOLID

Se realizó el rediseño del sistema de gestión de turnos aplicando los principios SOLID y ya cumpliendo con la Parte D de las consignas desde este punto (ver en el código):

- **SRP:** Cada clase tiene una única responsabilidad. La interfaz Notificacion define el contrato, y las clases concretas (EmailNotificacion, SmsNotificacion, WhatsappNotificacion) se encargan de cada tipo de notificación.
- **OCP:** El sistema está abierto para extensión (nuevos canales) sin modificar el código existente. Se utiliza abstracción mediante la interfaz Notificacion.
- **LSP (Liskov Substitution Principle):** Las implementaciones concretas pueden sustituir a la interfaz sin romper el sistema.
- **ISP (Interface Segregation Principle):** La interfaz Notificacion es minimalista, evitando sobrecarga de responsabilidades.
- **DIP:** GestorTurnos depende de la abstracción Notificacion, no de implementaciones concretas. Se aplica inyección de dependencias por constructor.
- **Resultado final:** El sistema está desacoplado, extensible y mantenido bajo los principios SOLID.

Parte C - Extensibilidad

Se agrega la interfaz Notificacion como abstracción base. Para cada nuevo canal, se crea una clase que implementa Notificacion:

- `TelegramNotificacion implements Notificacion`

- `PushNotificacion implements Notificacion`

Este diseño permite añadir estas funcionalidades sin modificar las clases existentes. Revisar el código para mayor precisión.

Parte D - Dependency Inversion Principle (DIP)

El Principio de Inversión de Dependencias (DIP) fue aplicado desde el comienzo del rediseño en la Parte B, organizando la aplicación desde un principio con abstracciones. Esto permitió:

- **Estructura modular:** La interfaz Notificacion se define primero como punto de abstracción.
- **Desacoplamiento inicial:** GestorTurnos depende exclusivamente de la abstracción, no de implementaciones concretas.
- **Extensibilidad futura:** Al implementar los nuevos medios de comunicación (Telegram, Push) en las Partes C y D, solo requieren nuevas implementaciones de Notificacion sin modificar GestorTurnos.
- **Resultado final:** La organización desde el inicio con abstracciones facilita la implementación de nuevos canales sin tocar el código existente.

Tabla con relaciones clave (Diagrama UML)

Tipo de Relación	Elementos Involucrados	Dirección	Cardinalidad	Descripción Técnica
Implementación	EmailNotificacion, SmsNotificacion, WhatsappNotificacion, TelegramNotificacion, PushNotificacion -> Notificacion	De las clases concretas hacia la interfaz	N:1	Las cinco clases concretas satisfacen el contrato definido por la interfaz Notificacion. Este es una implementación, que refuerza el Liskov Substitution Principle (LSP) y el Open/Closed Principle (OCP): cualquier clase concreta es intercambiable sin modificar a GestorTurnos.
Dependencia de Creación (y asociación)	GestorTurnos -> Notificacion	Desde la clase de alto nivel hacia la abstracción	1:1	La dependencia se materializa mediante inyección de dependencias vía constructor. GestorTurnos requiere una instancia de Notificacion para operar, pero no crea la instancia concreta (esta es proporcionada externamente en el momento de

Tipo de Relación	Elementos Involucrados	Dirección	Cardinalidad	Descripción Técnica
n)				instanciamiento). Se trata entonces de una dependencia de creación. El gestor invoca <code>Notificacion.enviar()</code> en tiempo de ejecución sin conocer la implementación concreta. Esto es la esencia del Dependency Inversion Principle (DIP): GestorTurnos depende de una abstracción, no de una implementación concreta.

Diagrama UML

